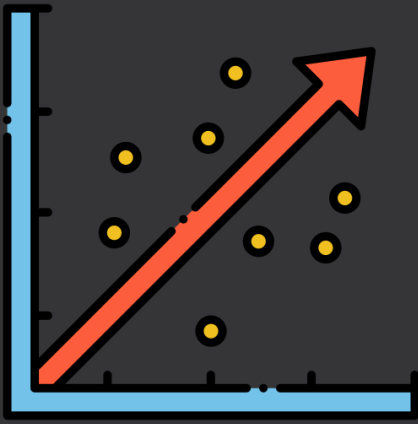




Simple Linear Regression

Lecture 8 – HCCDA-AI

Dr. Muhammad Sajjad

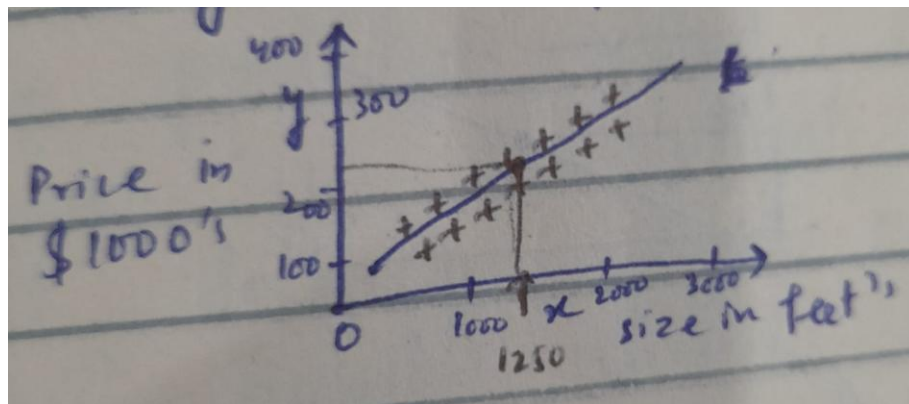
RA: Imran Nawar

Simple Linear Regression

- Linear Regression with one variable.
- A type of regression algorithms that models the relationship between a dependent variable and a single variable.

$$y = mx + c \quad \text{line equation / slope-intercept form}$$

- **Independent variables** → input features
- **Dependent variables** → What we want to predict
- Finding a line that fits the data
- The training set is a subset of your data on which your model will learn how to predict the dependent variable with the independent variables.

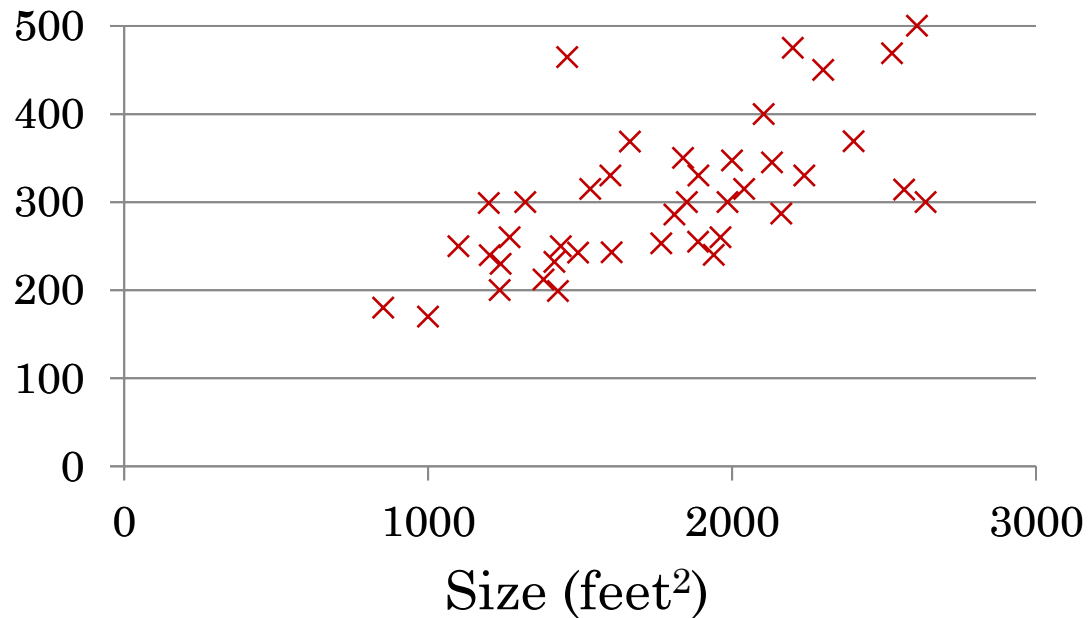


Linear regression with one variable

Model Representation

Housing Prices (Portland, OR)

Price
(in 1000s of
dollars)



Supervised Learning

Given the “right answer” for each example in the data.

Regression Problem

Predict real-valued output

Training set of housing prices (Portland, OR)

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

$m = 47$

Notation:

m = Number of training examples

$\mathbf{x}$'s = "input" variable / features

$\mathbf{y}$'s = "output" variable / "target" variable

(x, y) - one training example

$(x^{(i)}, y^{(i)})$ - i th training example

$$x^{(1)} = 2104$$

$$x^{(2)} = 1416$$

$$y^{(1)} = 460$$

Training Set



Learning Algorithm



Size of house



h



Estimated price

x

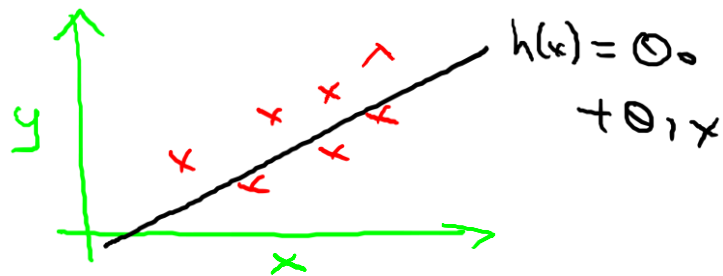
hypothesis

(estimated value of y)

h maps from x 's to y 's.

How do we represent h ?

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



Linear regression with one variable.
Univariate linear regression.

Linear regression with one variable

Cost Function

Training Set

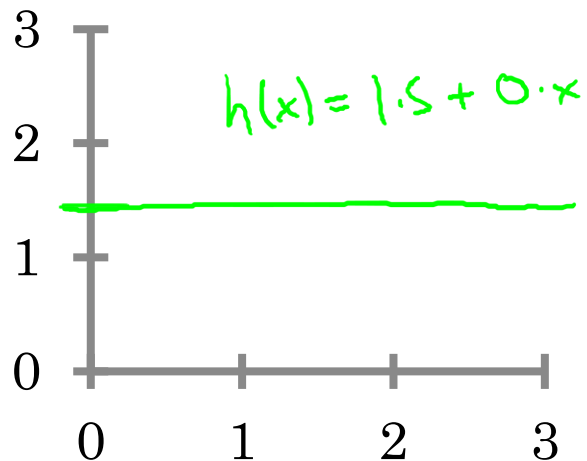
Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

θ_i 's: Parameters

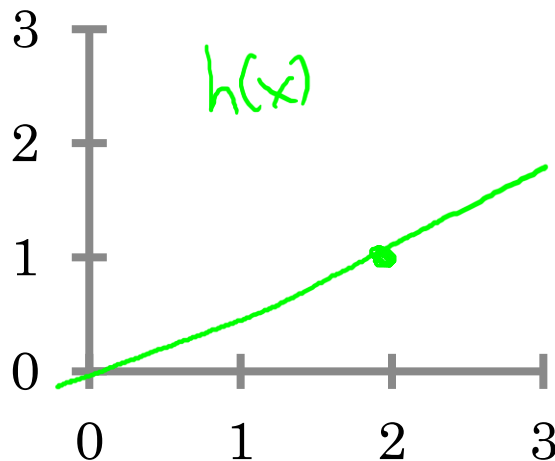
How to choose θ_i 's ?

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



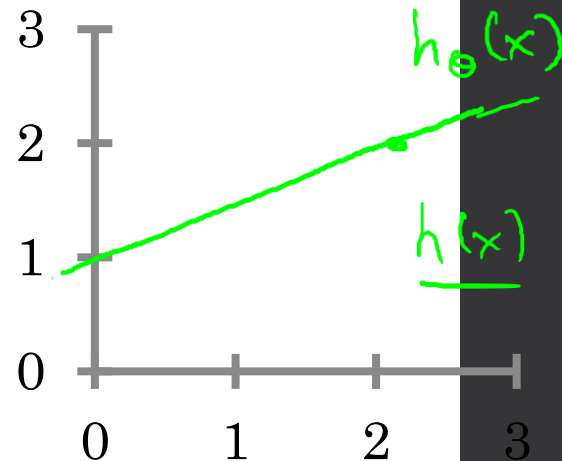
$$\theta_0 = 1.5$$

$$\theta_1 = 0$$



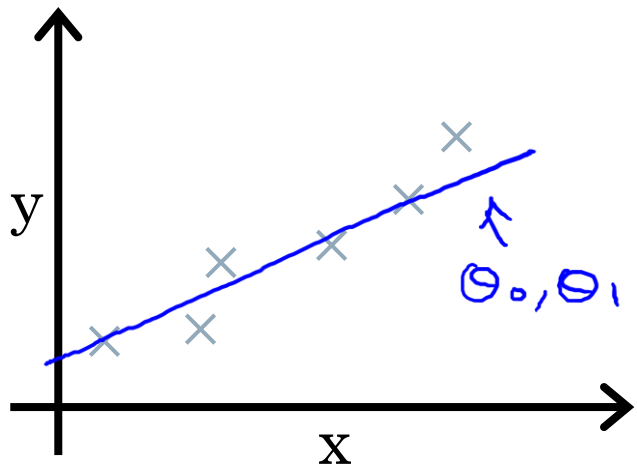
$$\theta_0 = 0$$

$$\theta_1 = 0.5$$



$$\theta_0 = 1$$

$$\theta_1 = 0.5$$



$$\text{minimize}_{\theta_0, \theta_1} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

training examples

$$h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$$

Idea: Choose θ_0, θ_1 so that $h_{\theta}(x)$ is close to y for our training examples (x, y)

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\text{minimize}_{\theta_0, \theta_1} \underbrace{J(\theta_0, \theta_1)}$$

Cost function
Squared error function

Cost Function

- Cost function represent the error between the actual value and the predicted value.
- A cost function, also known as a loss function or objective function, is a mathematical function that measures the difference between the predicted values of a model and the actual values in a dataset.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Cost function can be different based on different models. As previously discussed, this is the cost function for linear regression.
- Lesser the cost function value, the more accurate will be the prediction. Higher the cost function value the lesser accurate will be the prediction.

Linear regression with one variable

Cost function intuition I

Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$

Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal: minimize $J(\theta_0, \theta_1)$
 θ_0, θ_1

Simplified

$$h_{\theta}(x) = \theta_1 x$$

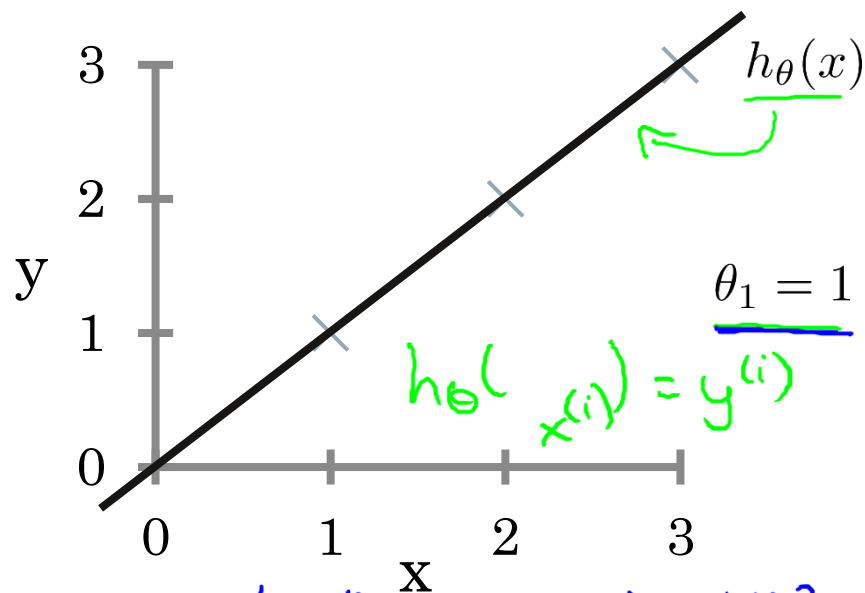
$$\theta_1$$

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

minimize $J(\theta_1)$
 θ_1

$$h_{\theta}(x)$$

(for fixed θ_1 , this is a function of x)

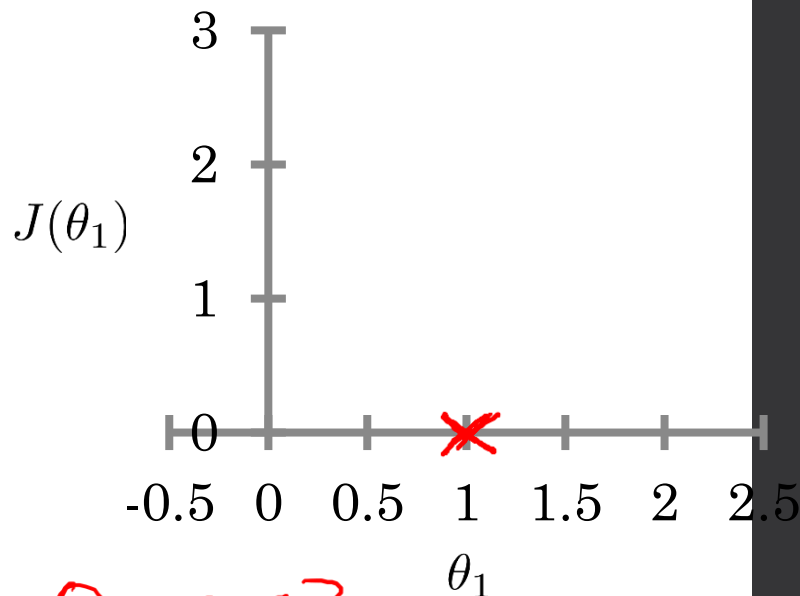


$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$= \frac{1}{2m} \sum_{i=1}^m (\theta_1 x^{(i)} - y^{(i)})^2 = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0^2$$

$$J(\theta_1)$$

(function of the parameter θ_1)

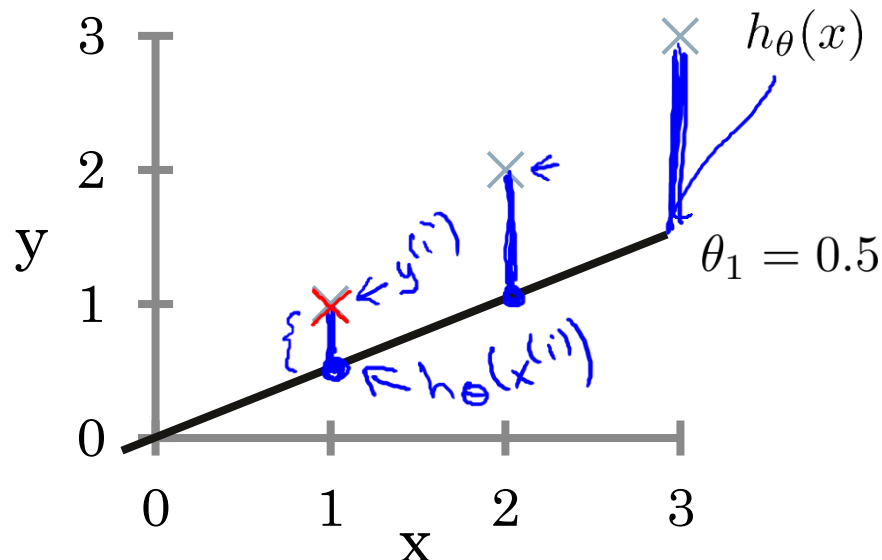


$$\theta_1 = 0.5?$$

$$J(1) = 0$$

$$h_{\theta}(x)$$

(for fixed θ_1 , this is a function of x)

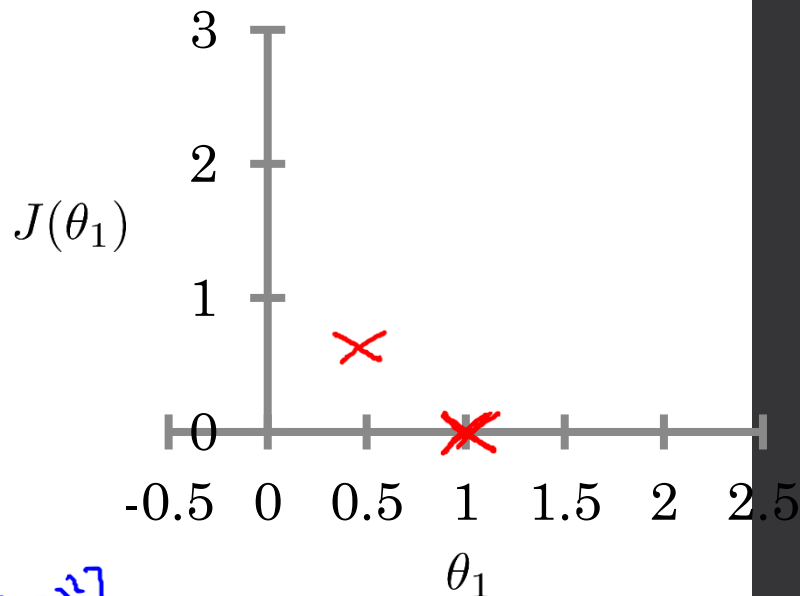


$$J(0.5) = \frac{1}{2m} [(0.5-1)^2 + (1-2)^2 + (1.5-3)^2]$$

$$= \frac{1}{2 \times 3} (3.5) = \frac{3.5}{6} \approx 0.58$$

$$J(\theta_1)$$

(function of the parameter θ_1)

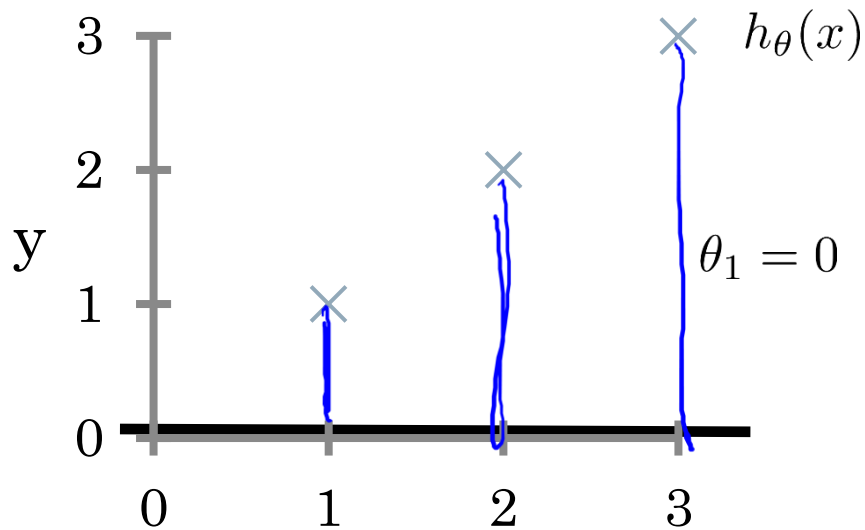


$$\theta_1 = 0?$$

$$J(0) = ?$$

$$h_{\theta}(x)$$

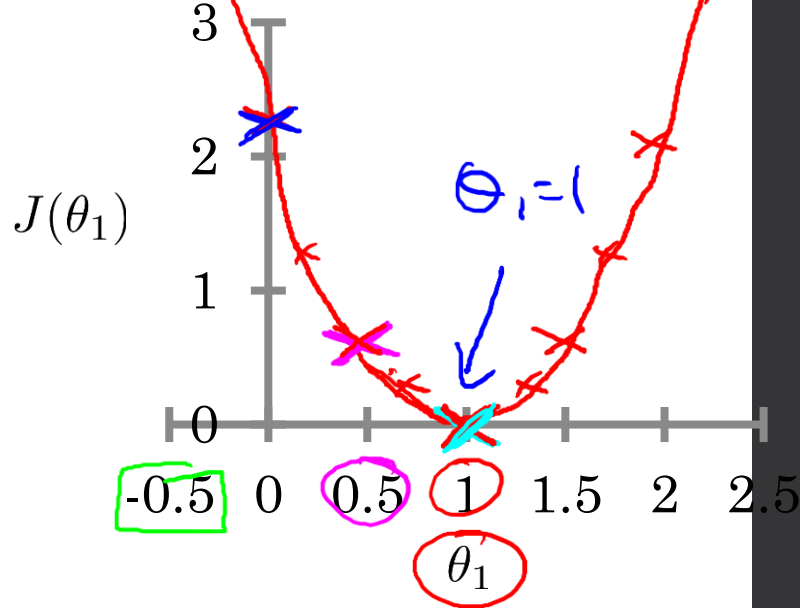
(for fixed θ_1 , this is a function of x)



$$J(0) = \frac{1}{2m} (1^2 + 2^2 + 3^2) \\ = \frac{1}{6} \cdot 14 \approx 2.3$$

$$J(\theta_1)$$

(function of the parameter θ_1)



$$\text{minimize}_{\theta_1} J(\theta_1)$$

minimize $J(\theta_1)$
 θ_1

- This is our objective function for linear regression.
- The optimization objective for our learning algorithm is we want to choose the value for θ , that minimize $J(\theta)$.

Types of Cost Functions:

The choice of cost function depends on the specific task and the type of model being trained. Here are some common types of cost functions:

1. Mean Squared Error (MSE):
 - Used in regression problems.
 - $MSE = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$
2. Mean Absolute Error (MAE):
 - Used in regression tasks.
 - $MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$
3. Binary Cross Entropy Loss:
 - Used in binary classification problems.
4. Categorical Cross Entropy Loss:
 - Used in Multiclass classification problems.

Linear regression with one variable

Cost function intuition II

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

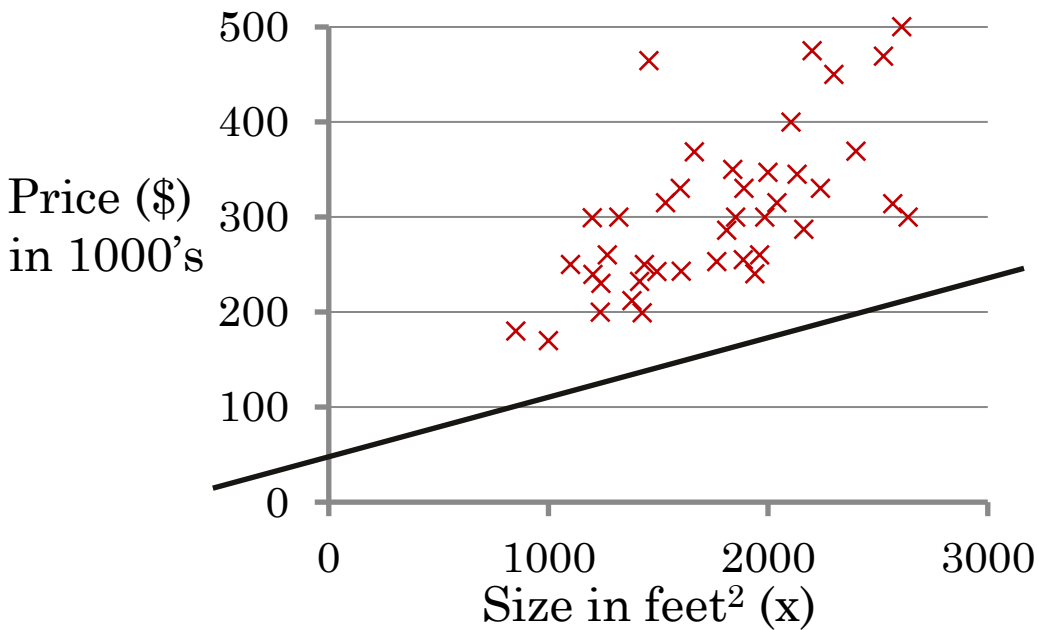
Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$

$$h_{\theta}(x)$$

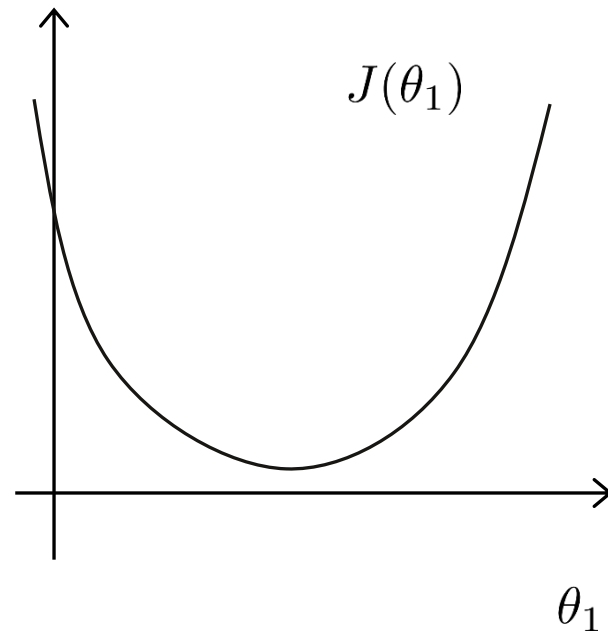
(for fixed θ_0, θ_1 , this is a function of x)



$$h_{\theta}(x) = 50 + 0.06x$$

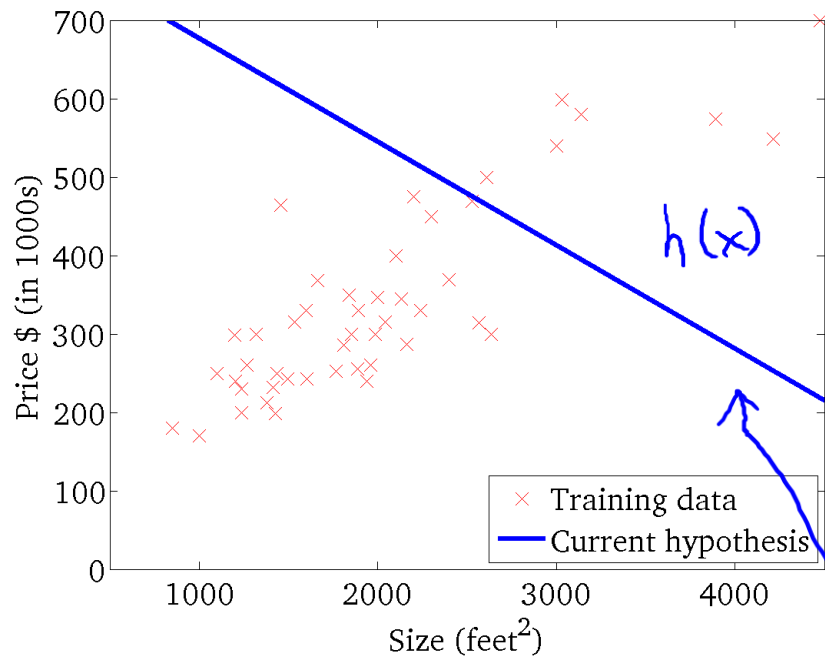
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



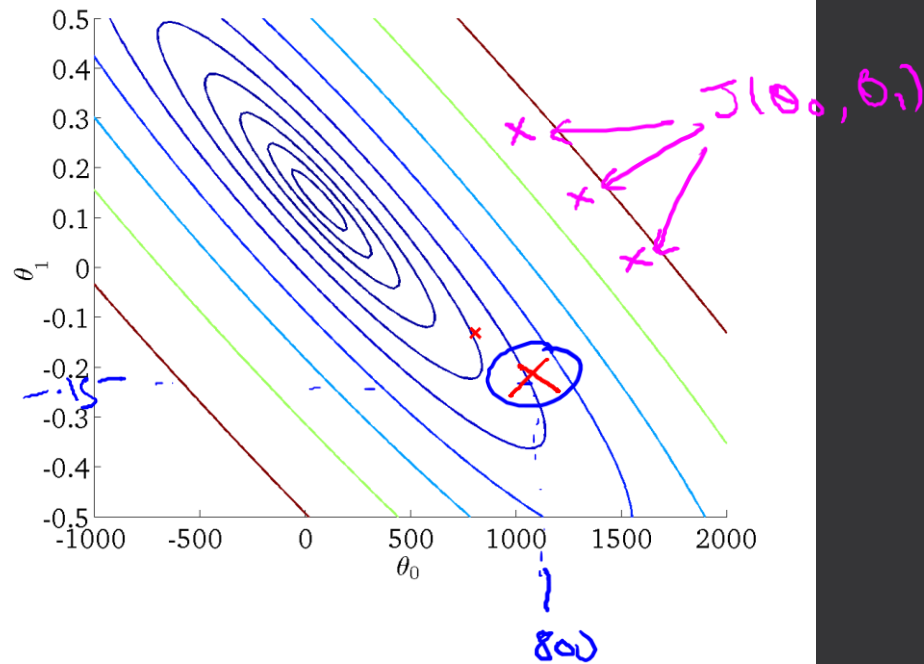
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



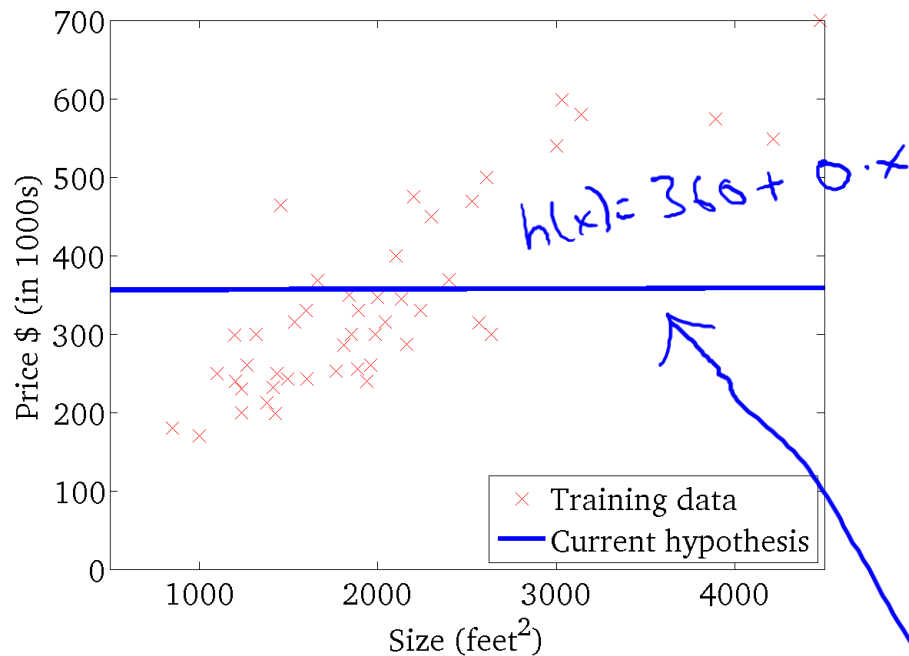
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



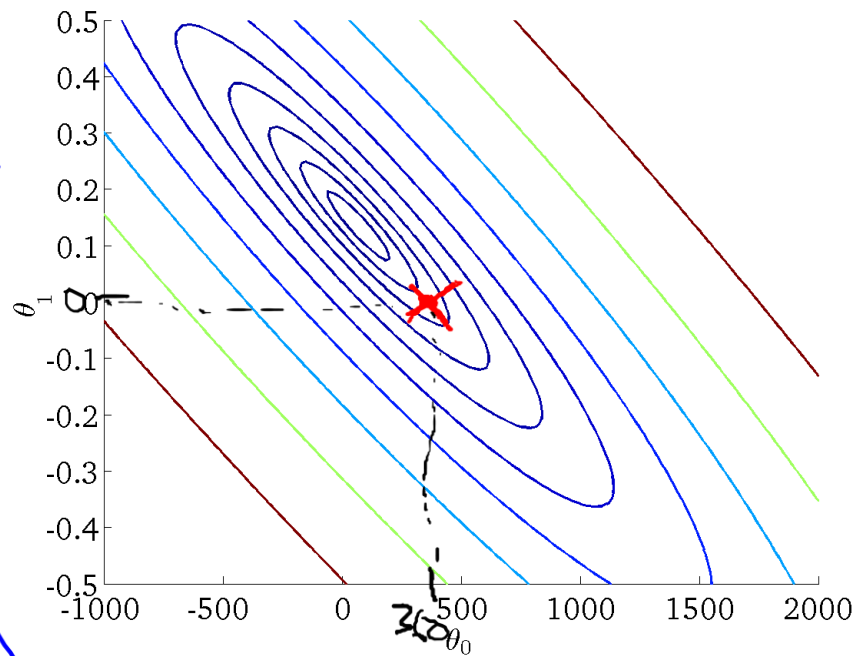
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

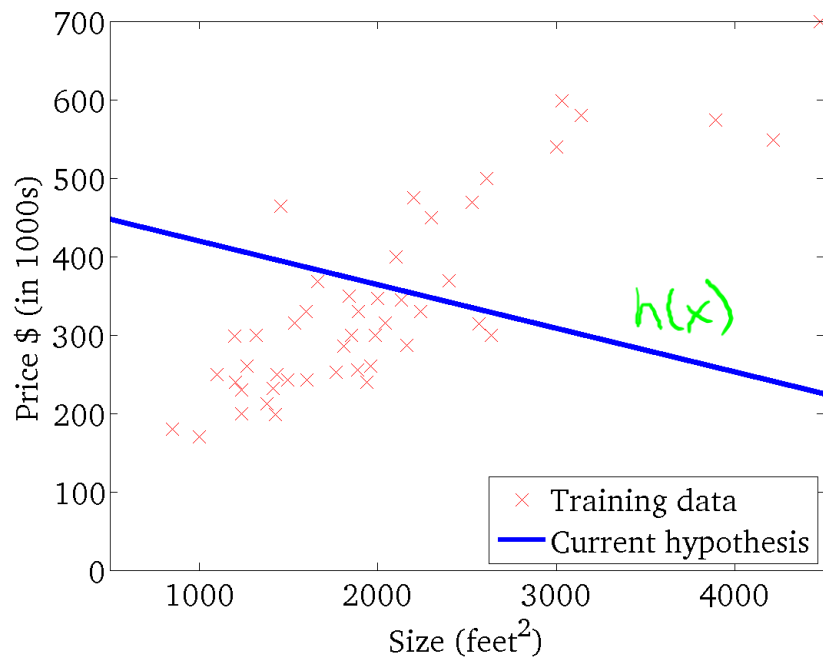
(function of the parameters θ_0, θ_1)



$$\begin{cases} \theta_0 = 360 \\ \theta_1 = 0 \end{cases}$$

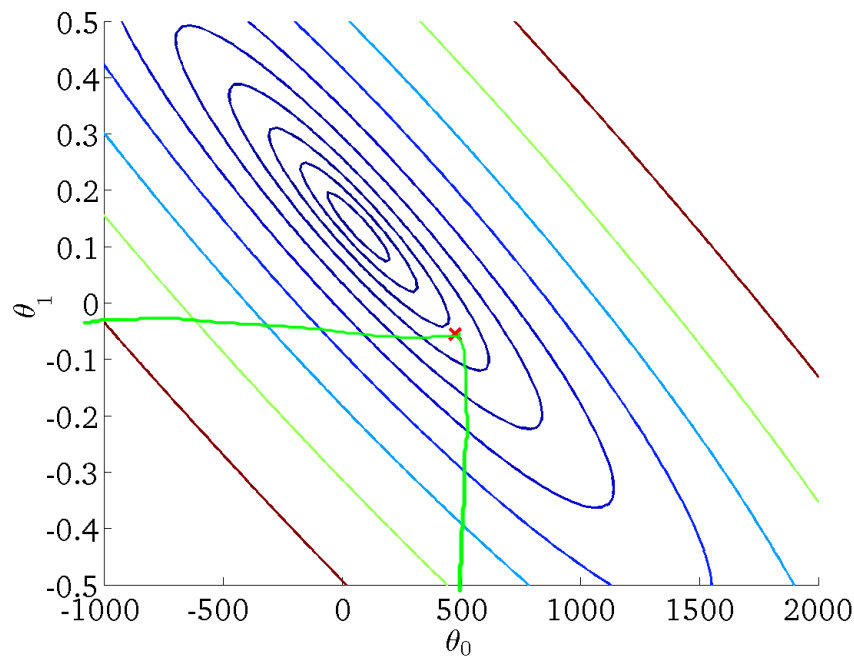
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



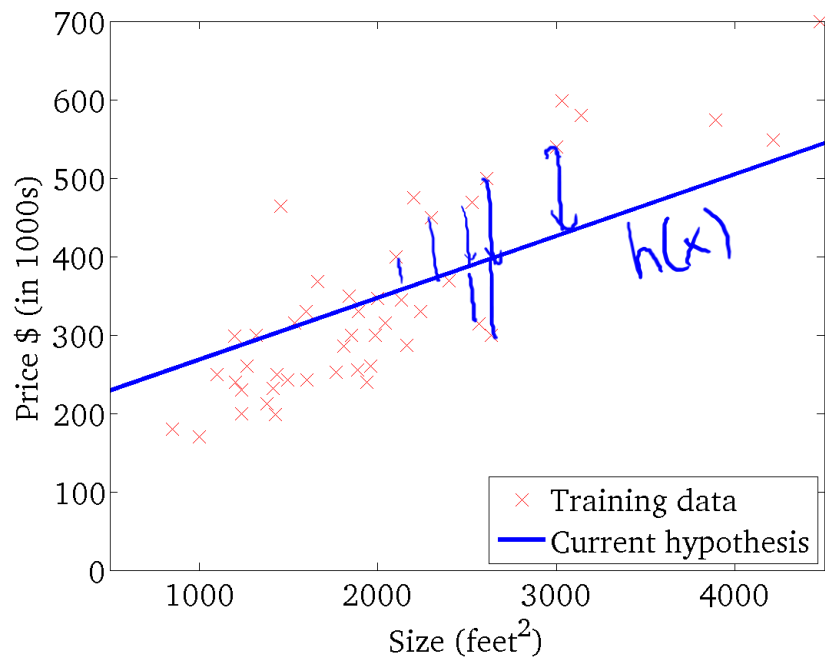
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



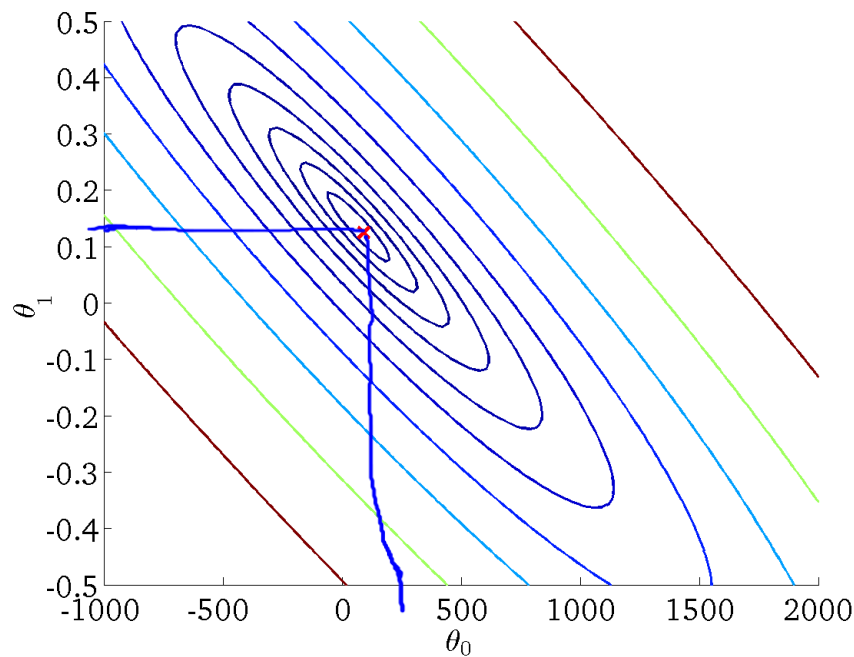
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



Minimizing the Cost Function

- To find the best-fit line, we need to minimize the cost function. This is done by adjusting model parameters to reduce the error.
- **Our goal:** Find values of θ_0 and θ_1 that minimize the cost function.
- We need a systematic approach to find these optimal parameters.
- **Key Question:** How do we adjust θ_0 and θ_1 to reduce our prediction errors?
- **Gradient Descent:** An iterative algorithm that steps towards the minimum.

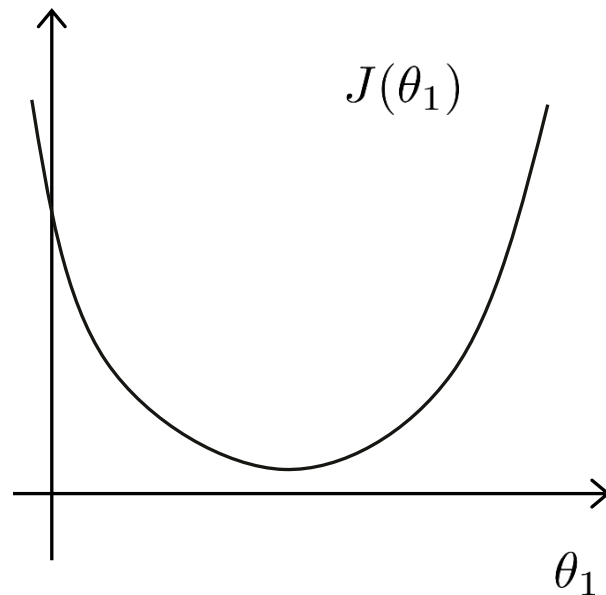
Next, we explore **Gradient Descent**, an optimization algorithm for this purpose.

Linear regression with one variable

Gradient Descent

Gradient Descent

- Gradient descent is an optimization algorithm used to minimize the cost function of a model by iteratively adjusting the parameters of the model.
- It is used to minimize the cost function value so that our straight line fits bests to the dataset.
- With the gradient descent we are taking steps to reach to it's minimum.

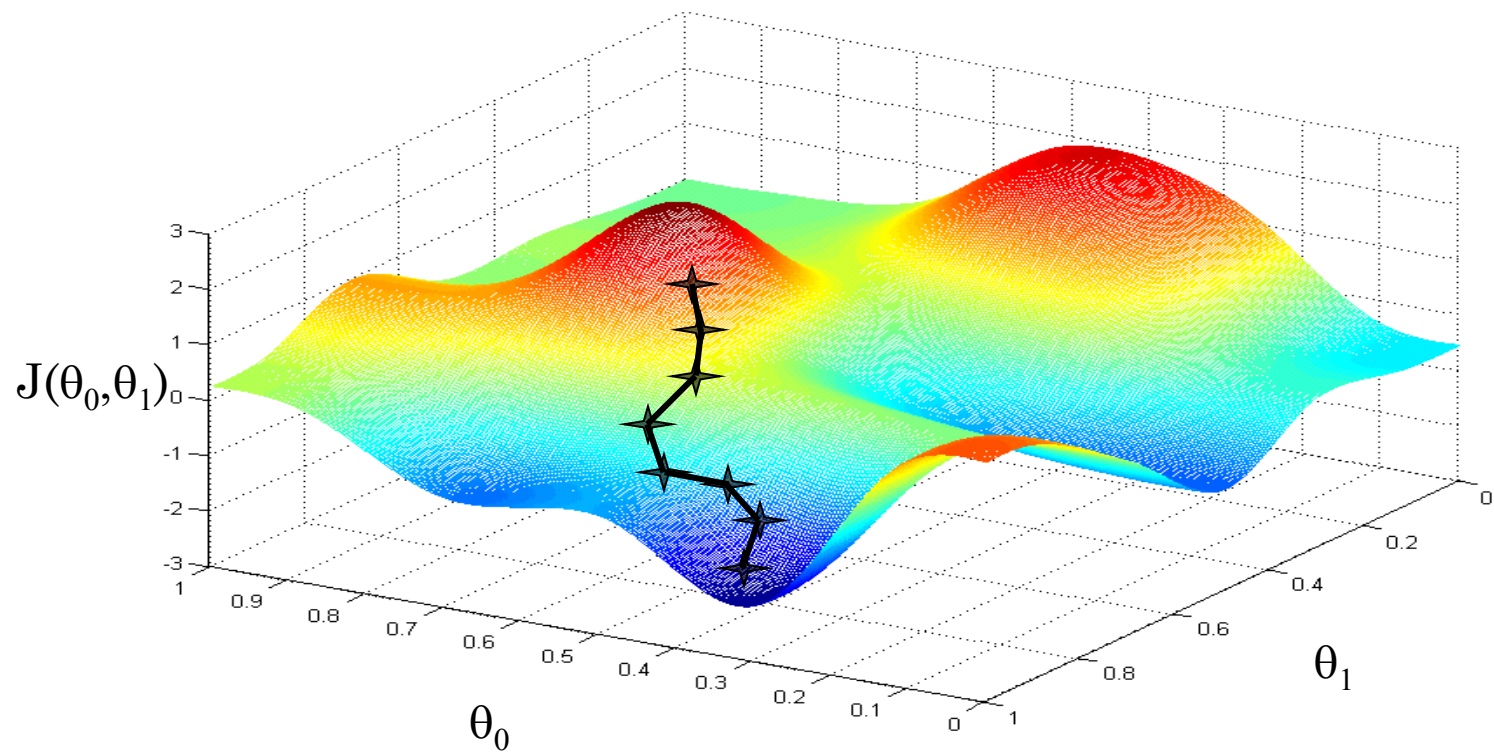


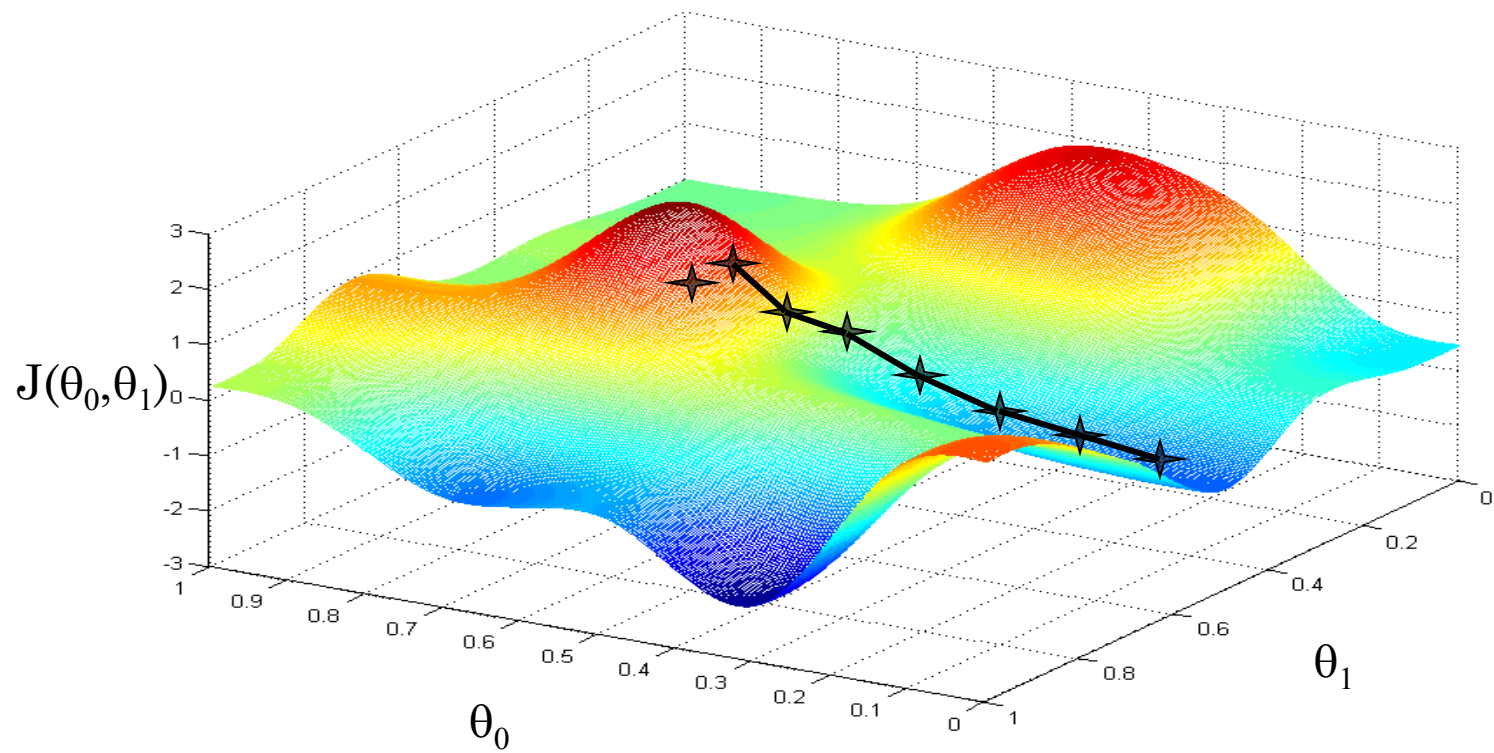
Have some function $J(\theta_0, \theta_1)$

Want $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Outline:

- Start with some θ_0, θ_1
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$ until we hopefully end up at a minimum





Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)
}

Correct: Simultaneous update

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_0 :=$  temp0  
 $\theta_1 :=$  temp1
```

Incorrect:

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
 $\theta_0 :=$  temp0  
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_1 :=$  temp1
```

Key Components of Gradient Descent

- $J(\theta_0, \theta_1)$ is the cost function that measures the difference between the predicted values and the actual values.
- α is the learning rate, which determines the size of step of each iteration.
- It is multiplied to scale the gradient. (α is a positive constant)
- The learning rate is a hyper parameter that needs to be chosen carefully. If it is too small, the algorithm may take a long time to converge, while if it is too large the algorithm may overshoot the minimum and fail to converge.
- $\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ is the partial derivative of the cost function J with respect to parameter θ_j
- The derivative of a function represents the rate at which the function's output (dependent variable) changes with respect to its input (independent variable).

Linear regression with one variable

Gradient Descent Intuition

Gradient descent algorithm

repeat until convergence {

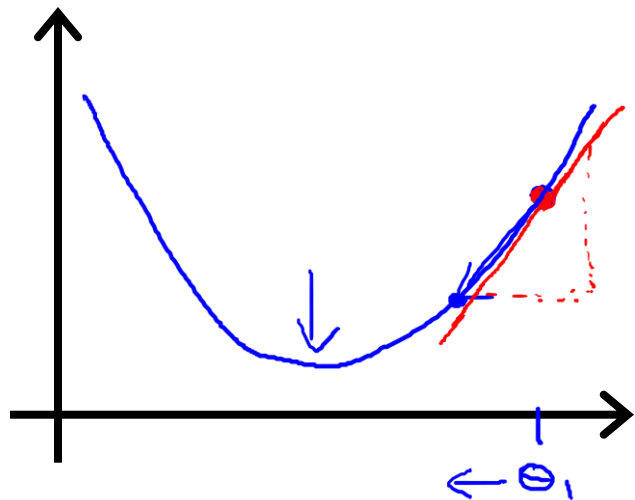
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

}

learning
rate

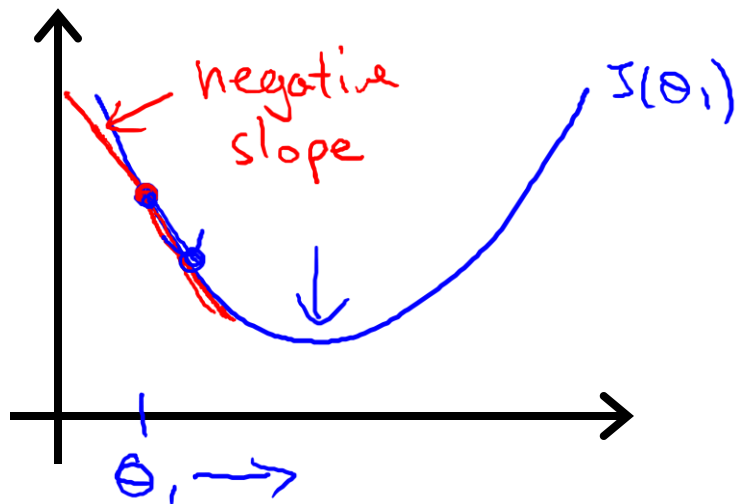
derivative

(simultaneously update
 $j = 0$ and $j = 1$)



$$\theta_1 := \theta_1 - \underbrace{2}_{\geq 0} \frac{\partial}{\partial \theta_1} J(\theta_1)$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{positive number})$$



$$\frac{\partial}{\partial \theta_1} J(\theta_1)$$

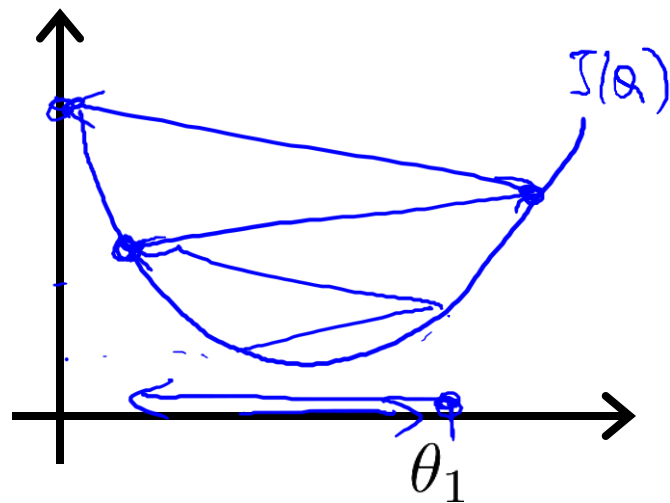
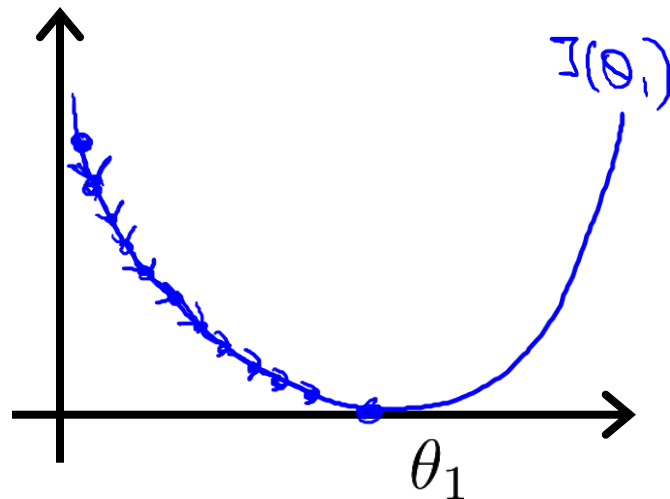
$$\leq 0$$

$$\theta_1 := \theta_1 - \alpha (\text{negative number})$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.



Linear regression with one variable

Gradient descent for linear regression

Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

Linear Regression Model

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{2}{2\theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{2}{2\theta_j} \frac{1}{2m} \sum_{i=1}^m (\underbrace{\theta_0 + \theta_1 x^{(i)}} - y^{(i)})^2\end{aligned}$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

Derivative of cost function in Linear Regression

- Figure out partial derivative with respect to θ_0, θ_1

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- In order for us to differentiate at a certain point we want to ignore the (Σ) summation sign and the value of m . Because we are finding the derivative at a single point.
- For finding the derivative at a composite function we typically apply the chain rule.

Composite Function:

$$F = f(g(x))$$

- **Chain rule:** The derivative of the outer function multiplied to the derivative of the inner function.

Derivative of cost function with respect to θ_0 :

$$\begin{aligned}\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_0} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m 2 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot (\theta_0^1 + \cancel{\theta_1 x^{(i)}} - \cancel{y^{(i)}})\end{aligned}$$

➤ $\theta_1 x^{(i)}$ and $y^{(i)}$ are constant items.

$$= \frac{1}{\cancel{2m}} \sum_{i=1}^m \cancel{2} (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot 1$$

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

Derivative of cost function with respect to θ_1 :

$$\begin{aligned}\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_1} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m 2 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot (\cancel{\theta_0^1} + \theta_1 x^{(i)} - \cancel{y^{(i)}})\end{aligned}$$

➤ θ_0 and $y^{(i)}$ are constant items.

$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient descent algorithm

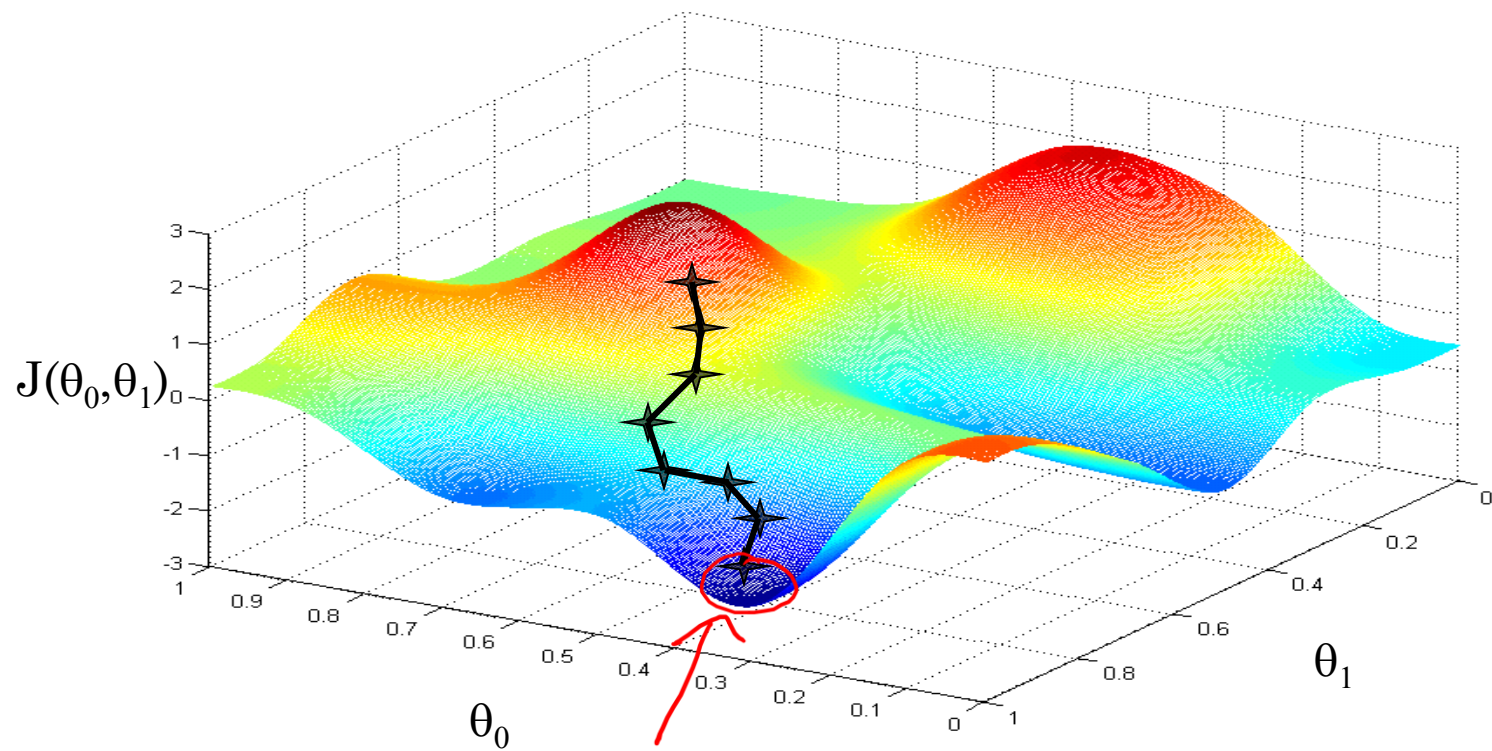
repeat until convergence {

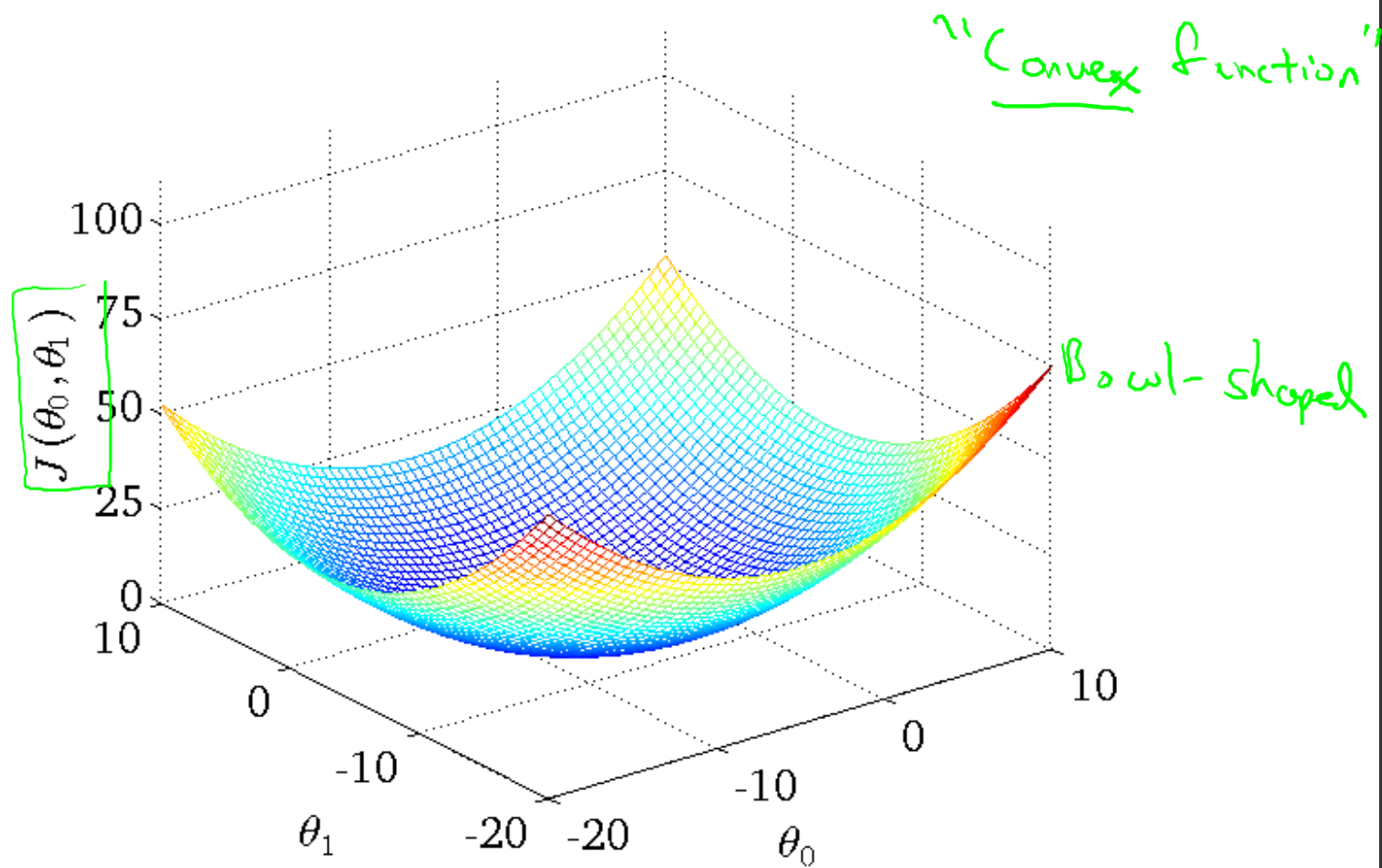
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

}
 update
 θ_0 and θ_1
 simultaneously

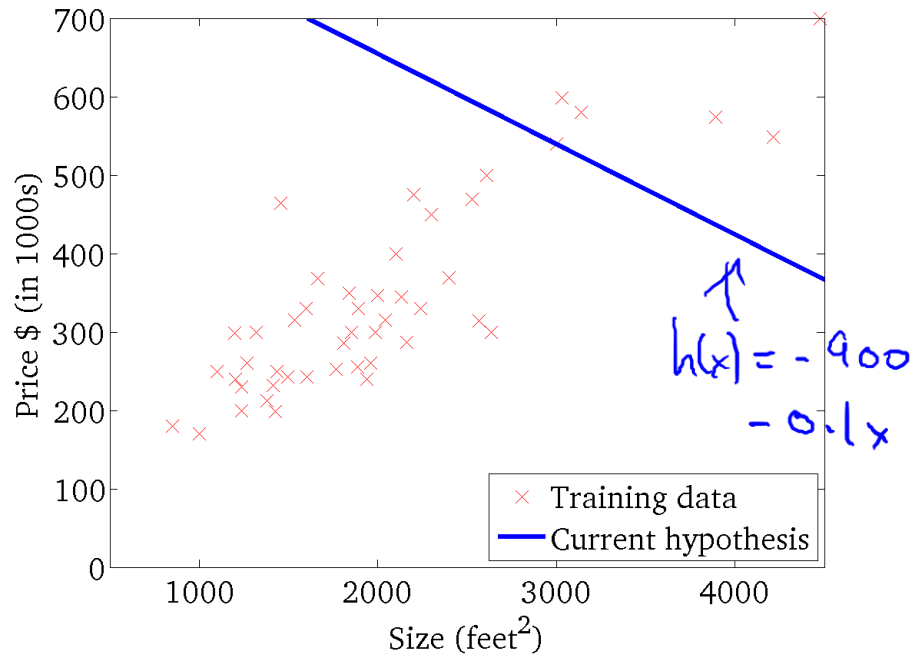
}





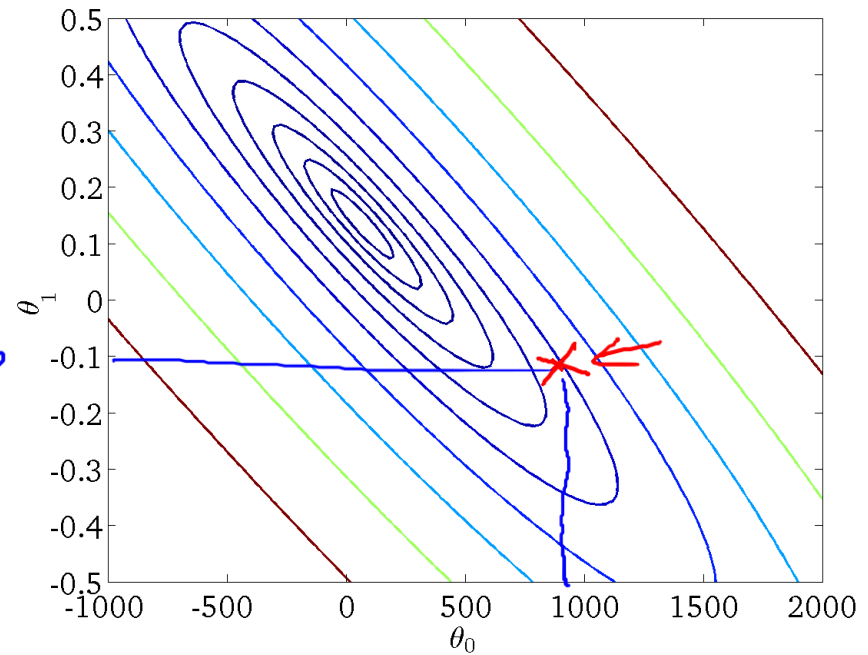
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



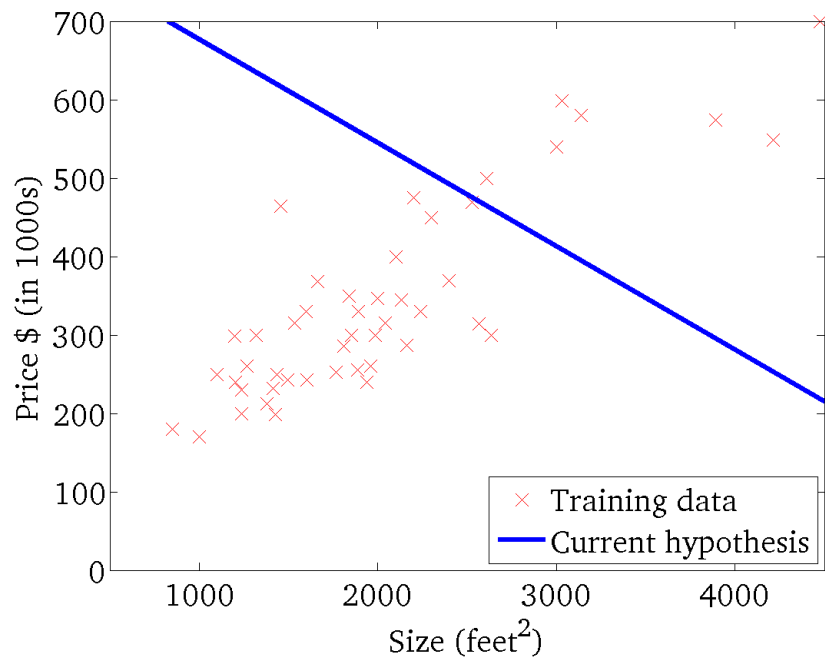
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



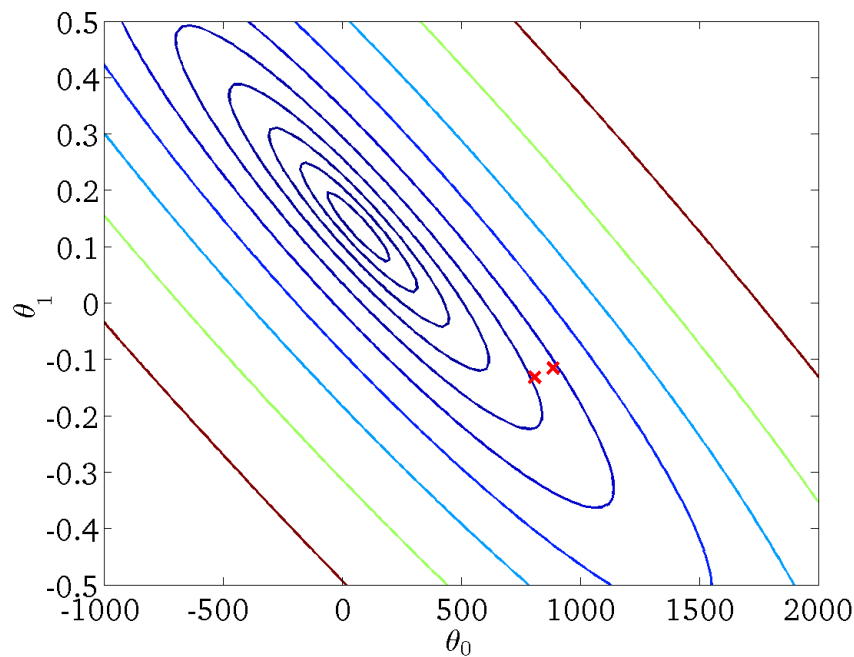
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



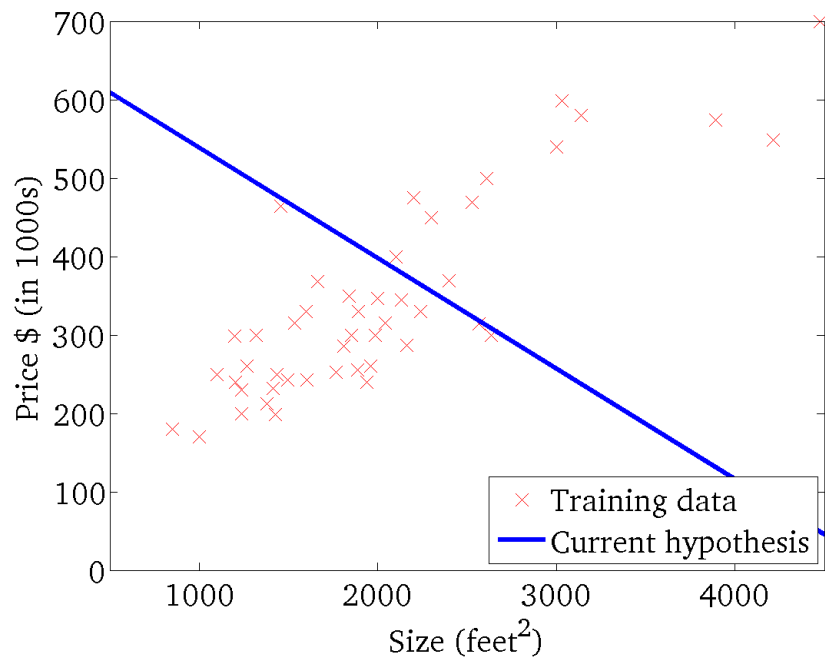
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



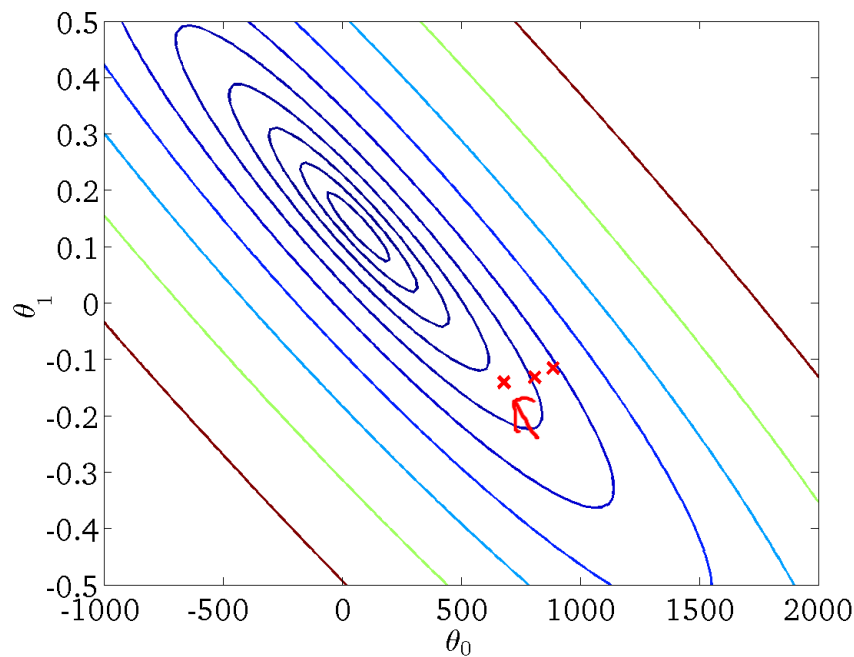
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



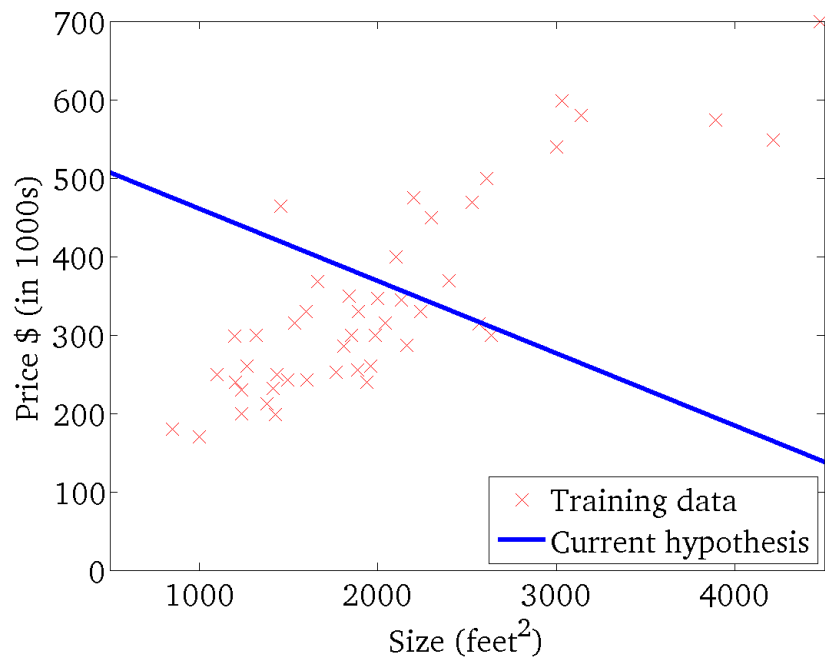
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



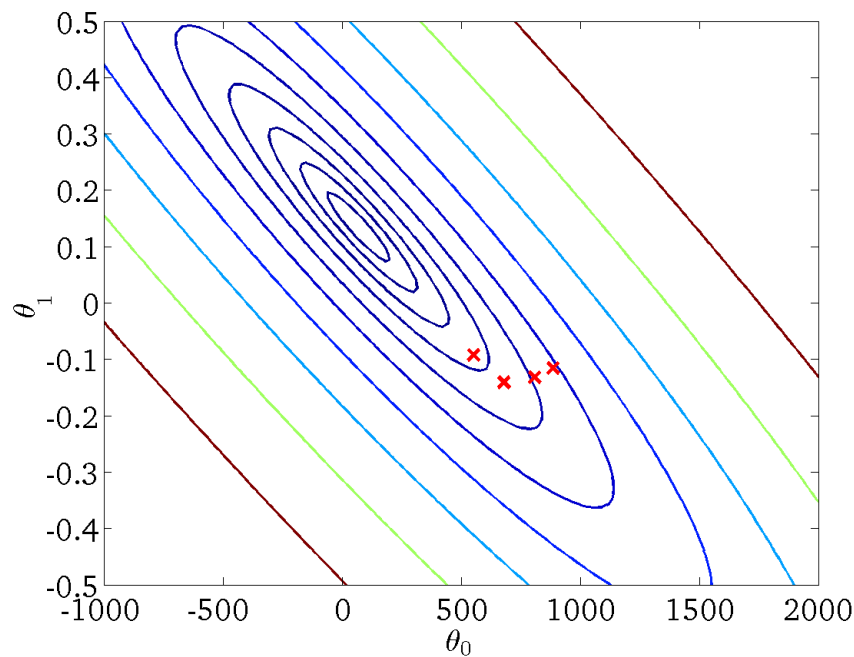
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



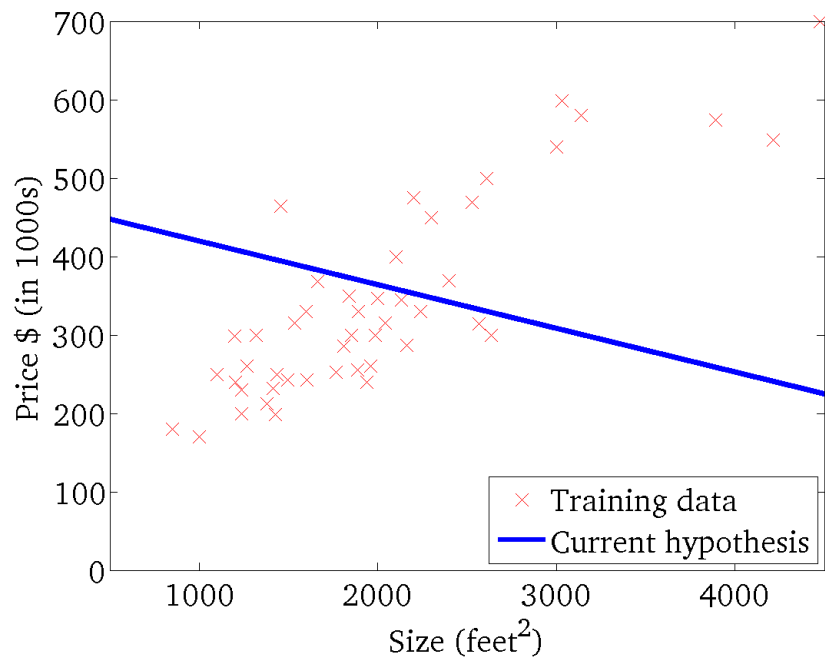
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



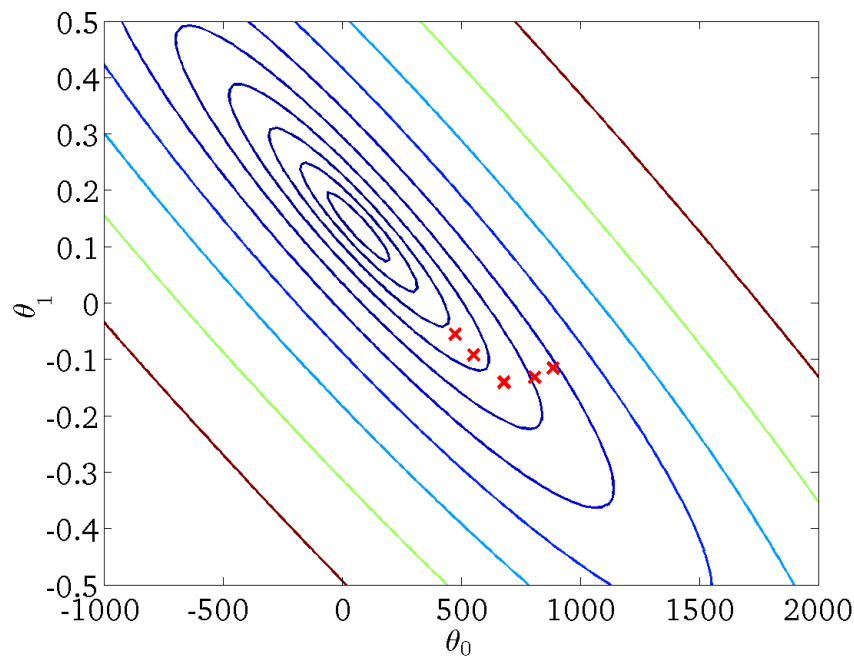
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



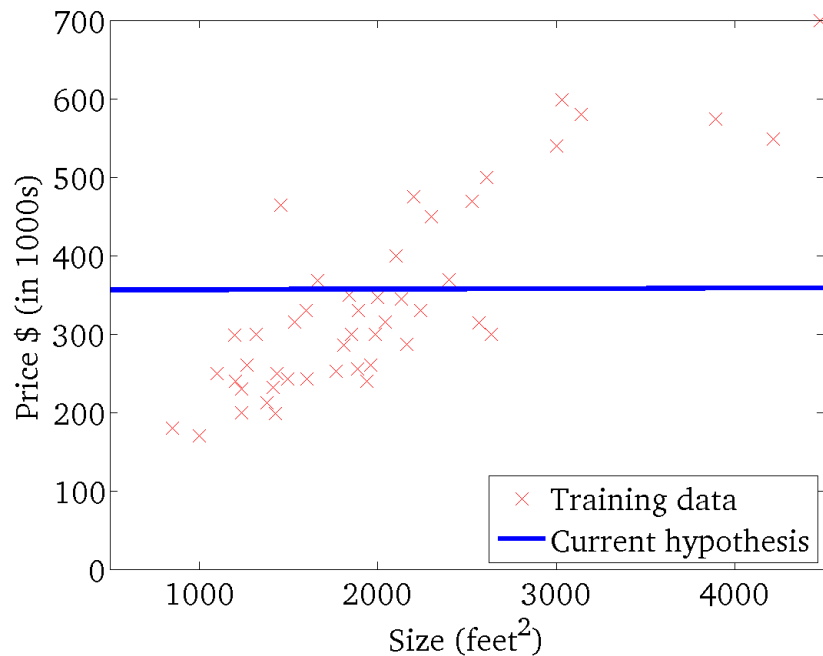
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



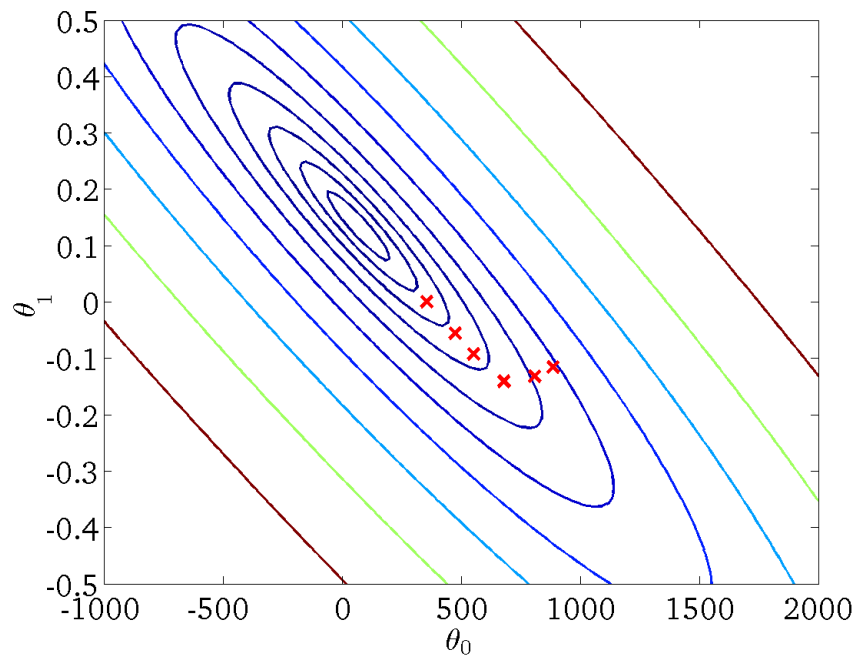
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



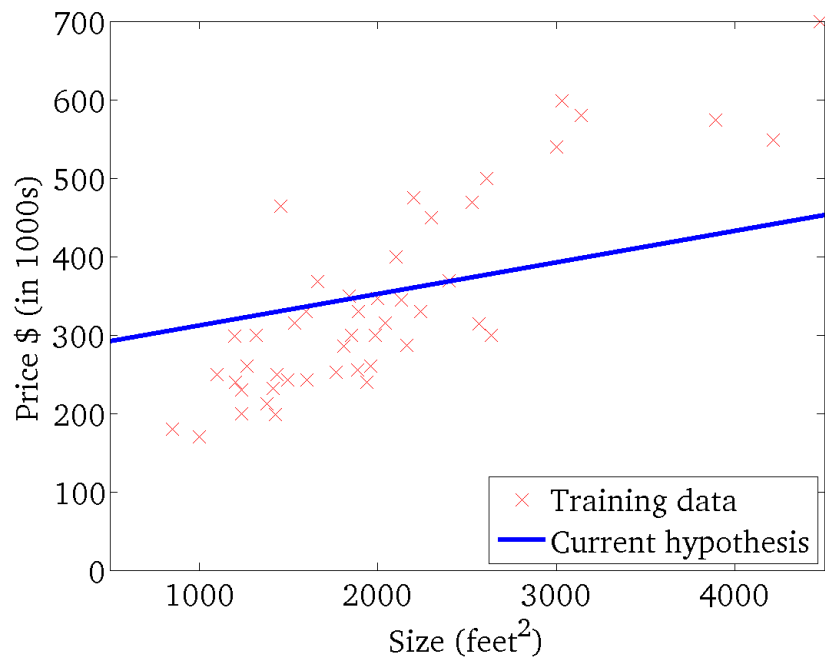
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



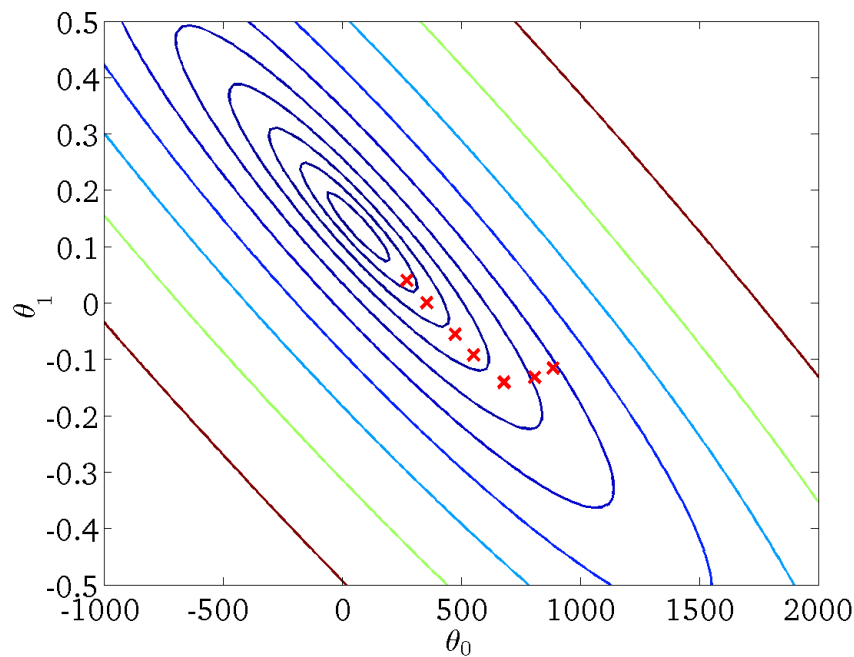
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



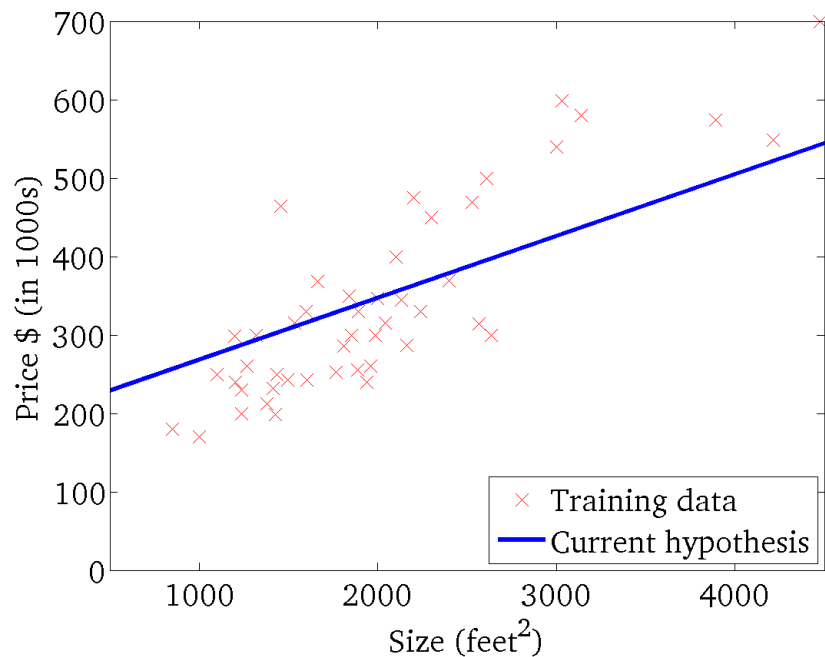
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



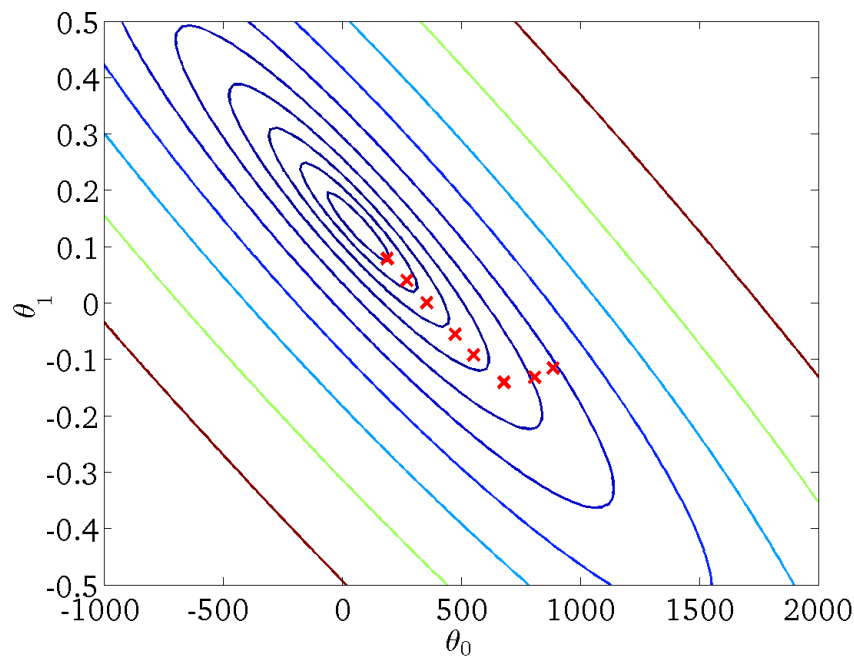
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



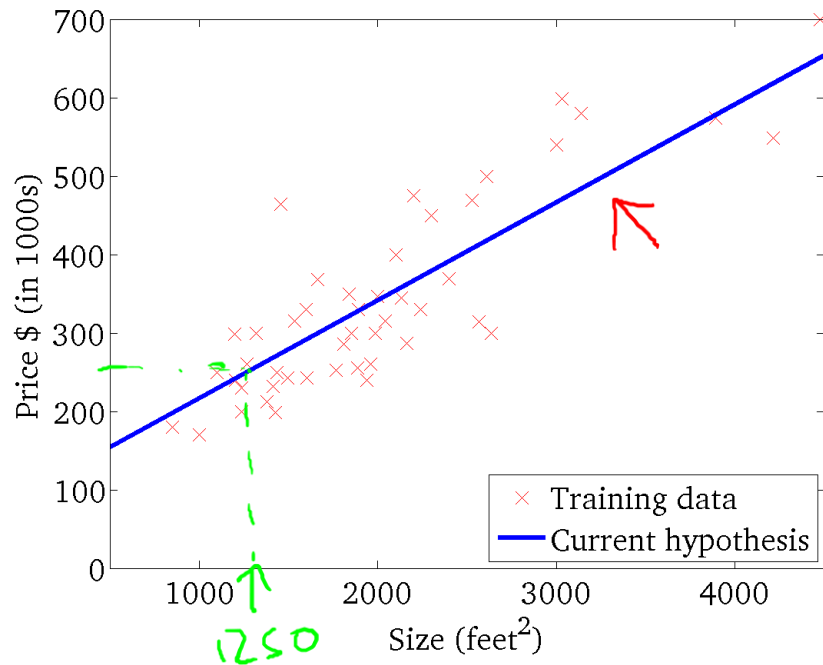
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



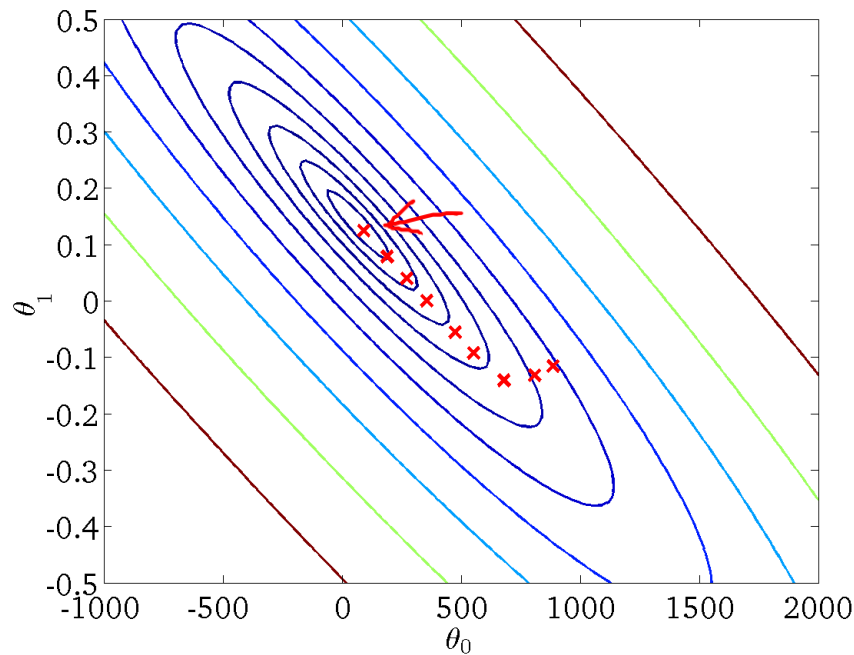
$$h_{\theta}(x)$$

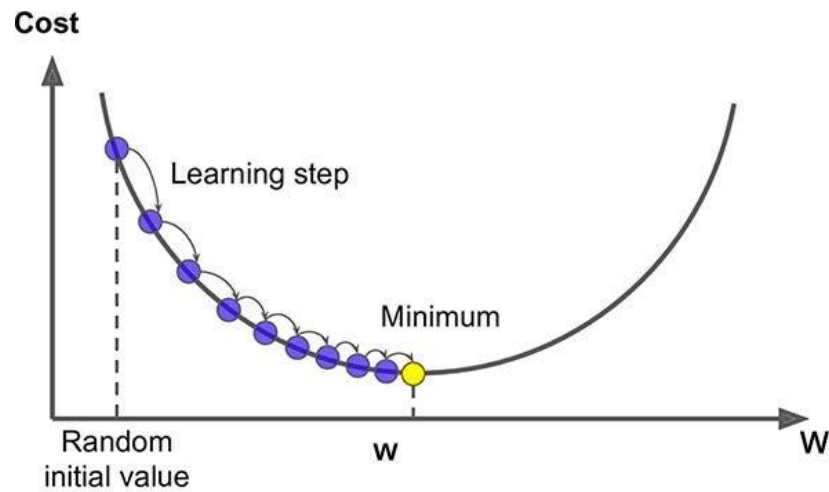
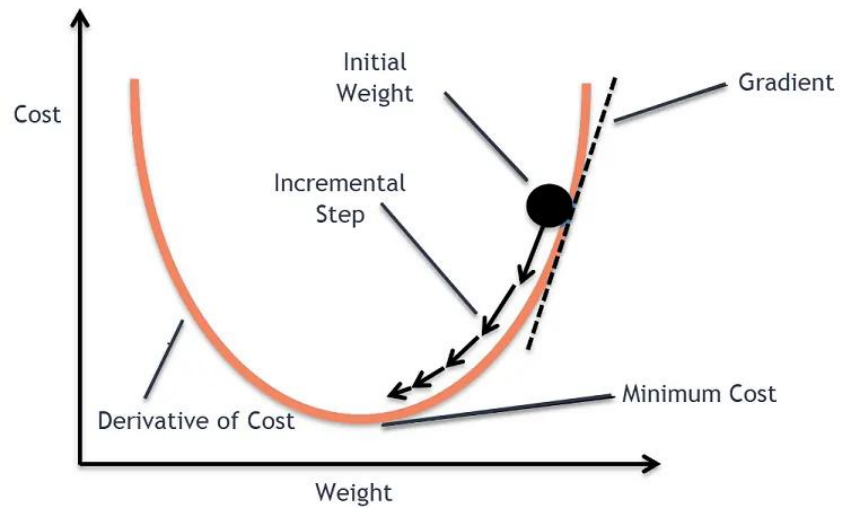
(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





Notable Gradient Descent Algorithms

- Different variations of Gradient Descent help optimize learning efficiency:

➤ Batch Gradient Descent (BGD):

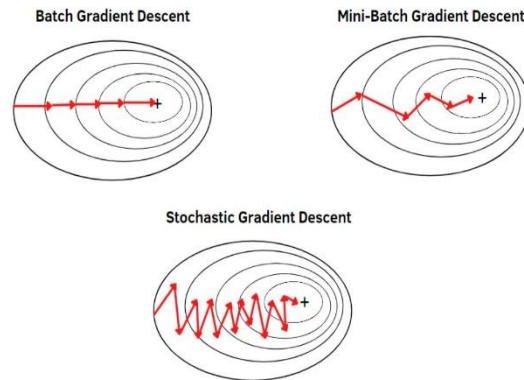
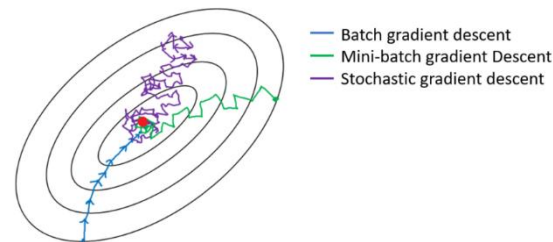
- Uses the entire dataset to compute gradients, ensuring stable updates but slower processing for large datasets.
- “Batch”: Each step of gradient descent uses all the training examples.
- BGD is computationally expensive since it requires processing the entire dataset in each iteration.

➤ Stochastic Gradient Descent (SGD):

- Updates parameters using one data point at a time, making it faster but more noisy in convergence.

➤ Mini-Batch Gradient Descent:

- A balance between BGD and SGD, updating parameters using small batches of data for efficient and stable learning.
- These variations impact **speed, stability, and accuracy** in training models.



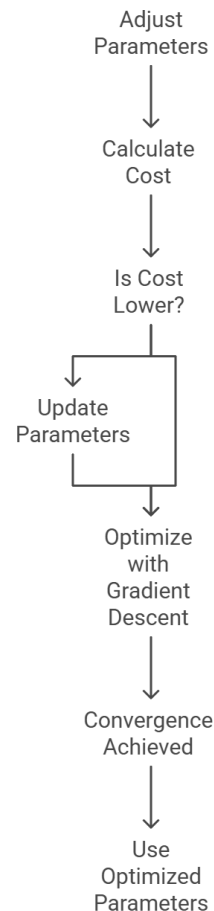
How do Models Learn?

- Machine Learning models learn by adjusting their parameters to minimize the cost function, which measures the difference between predicted and actual values.

- Cost Function (Mean Squared Error):**

$$\text{cost} = (y - \hat{y})^2 = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- A lower cost indicated better predictions.
- To minimize cost function, machine learning algorithms use optimization techniques like gradient descent.
- Optimization with Gradient Descent:**
 - Iteratively updates parameters in the direction that reduces cost.
 - Continues until convergence to (or near) the optimal values.
- Once trained, the model uses these optimized parameters to make predictions based on new data.





Thank You